Parameterizing SQL Queries – Protecting Against SQL Injection

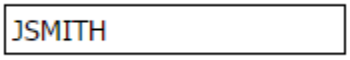
Lately, I have been observing an alarming trend in many of the scripts I look at in user systems: namely non-parameterized database queries.

Basically, the pattern looks like this…

```
// Imagine that the active document is an e-form, where a user can enter data in fields
string fieldValue = args.Document.EForm.Fields.Find("fieldName").Value;
var connection = app.Configuration.GetConnection("MyDatabaseConnection");
var command = connection.CreateCommand();
command.CommandText = string.Format("SELECT column1 FROM table WHERE column2 = '{0}'",
    fieldValue);
var reader = command.ExecuteReader();
while (reader.Read())
{
    // Do something with the results
}
```

We're pasting a value that a user can modify directly into a SQL query without examining its effects.

Now, in a normal scenario (where our user is insufficiently savvy or nefarious to try injecting SQL), nothing bad happens. For example, let's imagine that the database column is expecting a user name. If the user enters a normal value, like "JSMITH" …



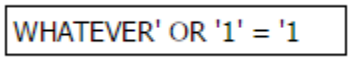
Then the SQL query generated in the background will be perfectly safe …

```
SELECT column1 FROM table WHERE column2 = 'JSMITH'
```

This will return only the expected row(s)

But what if the user is sufficiently savvy and evil?

For example, the user might try to get back extra information like this …



Then the SQL query generated in the background will be unsafe …

```
SELECT column1 FROM table WHERE column2 = 'WHATEVER' OR '1' = '1'
```

This will return every row in the table, because the "`OR '1' = '1'`" clause meant that the condition will always return true.

This is pretty bad, especially if you're dealing with healthcare, finances, or personal information, where the smallest data breach can result in enormous damage done to people and in huge fines levied against the client.

So clearly, this is something we want to prevent.

At this point, you might decide to just escape single-quotes, which would help with that specific example, but there are other ways for the sneaky user to inject SQL.

For example, a user might actually try to include additional SQL queries.

Here, our bad guy is attempting to damage the database itself…

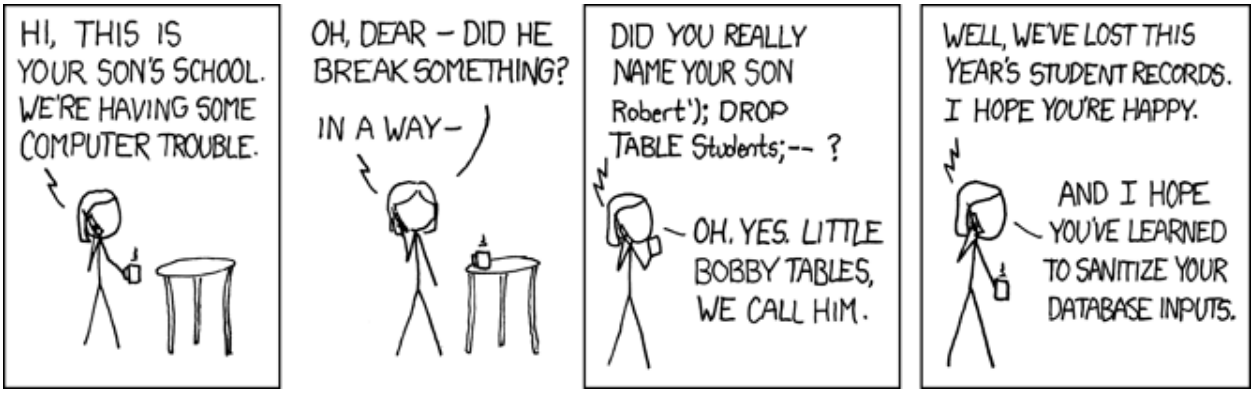
```
WHATEVER'; DROP TABLE some_table --
```

This will result in the following SQL query being generated…

```
SELECT column1 FROM table WHERE column2 = 'WHATEVER'; DROP TABLE some_table --'
```

Our user was even clever enough to comment out the closing single-quote, and now the executed query will destroy data, requiring a costly repair (and even that assumes that we have a recent backup).

Momentary humor interlude:



https://xkcd.com/327/

And yes, before you ask, in this case, escaping single-quotes would help, but what if we're querying a numeric field and wind up with…

```
SELECT column1 FROM table WHERE column2 = 0; DROP TABLE some_table
```

Because the user entered this…

```
0; DROP TABLE some_table
```

Fat lot of good escaping single-quotes does here…

So now, you might think that handling semicolons and double-hyphens could help. But many other mechanisms and control characters can be used. If you read the cartoon, you're also considering sanitizing against back-ticks and parentheses. Eventually, this can get a bit out of hand as you try to handle every conceivable bad character.

What if, instead of trying to include handlers for every potentially threatening character, you could just take advantage of an existing mechanism that sanitizes the input for you?

Enter parameterization!

By placing your values into the query as parameters, you can allow the .NET framework to properly sanitize values to prevent SQL injection.

We can repeat the same code block, this time using a parameter instead of plugging the field value directly into the SQL.

```
// Imagine that the active document is an e-form, where a user can enter data in fields
string fieldValue = args.Document.EForm.Fields.Find("fieldName").Value;
var connection = app.Configuration.GetConnection("MyDatabaseConnection");
var command = connection.CreateCommand();
command.CommandText = "SELECT * FROM some_table WHERE some_column = @fieldParam";
var fieldParam = command.CreateParameter();
fieldParam.ParameterName = "fieldParam";
fieldParam.DbType = DbType.String;
fieldParam.Value = fieldValue;
command.Parameters.Add(fieldParam);
var reader = command.ExecuteReader();
while (reader.Read())
{
    // Do something with the results
}
```

Now this bad example…

```
WHATEVER' OR '1' = '1
```

… results in this SQL query

```
SELECT column1 FROM table WHERE column2 = 'WHATEVER'' OR ''1'' = ''1'
```

This will not return anything, because it's actually looking for a single value "WHATEVER' OR '1' = '1'"

Likewise, this bad example will be treated the same…

```
WHATEVER'; DROP TABLE some_table --
```

```
SELECT column1 FROM table WHERE column2 = 'WHATEVER''; DROP TABLE some_table --'
```

Of course, nothing is 100% sure to protect your database from the bad stuff out there, but parameterizing queries protects against almost all forms of SQL injection.

The lesson, of course, is **never place a non-parameterized value into a SQL query in your code!**

Parameterization is easy to implement, and although it does have a performance cost (since the server has to prepare the SQL statement), the benefits are too great to ignore.

I'll point out that I have illustrated parameterization as a fancy way to handle character escapes. Although that is a big piece of the intent, parameterization does more than just that to keep your queries safe. Take advantage of them.

There is a less common scenario where you might be using input data to identify a database table or to specify a column in the table.

Here's an example of the wrong way to do this:

```
string tableName = args.Document.EForm.Fields.Find("fieldName").Value;
var connection = app.Configuration.GetConnection("MyDatabasConnection");
var command = connection.CreateCommand();
command.CommandText = "SELECT * FROM " + tableName + " WHERE some_column = 'some_value'";
var reader = command.ExecuteReader();
while (reader.Read())
{
    // Do something with the results
}
```

Since parameters can only be used to replace column values, not table or column names, parameterization is not the solution. But we still need to sanitize the input data so that something like this can't be used to wreak havoc.

```
keyitem101; DROP TABLE itemdata --
```

In an instance like this, we can still sanitize the data using the DbCommandBuilder class, like this…

```
string tableName = args.Document.EForm.Fields.Find("fieldName").Value;
var connection = app.Configuration.GetConnection("MyDatabaseConnection");
var command = connection.CreateCommand();
var factory = DbProviderFactories.GetFactory(connection);
var builder = factory.CreateCommandBuilder();
string sanitizedTableName = builder.QuoteIdentifier(tableName);
command.CommandText = "SELECT * FROM " + sanitizedTableName + " WHERE some_column = 'some_value'";
var reader = command.ExecuteReader();
while (reader.Read())
{
    // Do something with the results
}
```

Sure, it's a little convoluted, but it beats leaving vulnerabilities (and liabilities) in your code.

On the following pages, you'll find example code using several of the common connection types.

Keep in mind, these are only examples, and I have not included try/catch/finally blocks or other best practices.

Meanwhile, Happy Coding!

```
// SQL Server Data Sanitation Example

using System.Data;
using System.Data.SqlClient;
using Hyland.Unity;

namespace UnityScripts
{
    public class SqlServerParameterizationExample : IWorkflowScript
    {
        public void OnWorkflowScriptExecute(Application app, WorkflowEventArgs args)
        {
            string docId = args.Document.EForm.Fields.Find("Related Doc Id").Value;
            string tableName = args.Document.EForm.Fields.Find("Search Table").Value;
            using (var conn = (SqlConnection) app.Configuration.GetConnection("MyDbConnection"))
            {
                conn.Open();
                var builder = new SqlCommandBuilder();
                string sanitizedTableName = builder.QuoteIdentifier(tableName);
                string sql = "SELECT keyvaluechar FROM " + sanitizedTableName + " WHERE itemnum = @docId";
                using (var cmd = new SqlCommand(sql, conn))
                {
                    cmd.Parameters.Add(new SqlParameter
                    {
                        ParameterName = "docId",
                        SqlDbType = SqlDbType.BigInt,
                        Value = long.Parse(docId)
                    });
                    var reader = cmd.ExecuteReader();
                    while (reader.HasRows && reader.Read())
                    {
                        // Do something with the data
                    }
                }
                conn.Close();
            }
        }
    }
}
```

```
// Oracle Data Sanitation Example

using Hyland.Unity;
using Oracle.DataAccess.Client;

namespace UnityScripts
{
    public class OracleParameterizationExample : IWorkflowScript
    {
        public void OnWorkflowScriptExecute(Application app, WorkflowEventArgs args)
        {
            string docId = args.Document.EForm.Fields.Find("Related Doc Id").Value;
            string tableName = args.Document.EForm.Fields.Find("Search Table").Value;
            using (var conn = (OracleConnection) app.Configuration.GetConnection("MyDbConnection"))
            {
                conn.Open();
                var builder = new OracleCommandBuilder();
                string sanitizedTableName = builder.QuoteIdentifier(tableName);
                string sql = "SELECT keyvaluechar FROM " + sanitizedTableName + " WHERE itemnum = :1";
                using (var cmd = new OracleCommand(sql, conn))
                {
                    cmd.Parameters.Add(new OracleParameter
                    {
                        OracleDbType = OracleDbType.Int64,
                        Value = long.Parse(docId)
                    });
                    var reader = cmd.ExecuteReader();
                    while (reader.HasRows && reader.Read())
                    {
                        // Do something with the data
                    }
                }
                conn.Close();
            }
        }
    }
}
```

```
// ODBC Data Sanitation Example

using System.Data.Odbc;
using Hyland.Unity;

namespace UnityScripts
{
    public class OdbcParameterizationExample : IWorkflowScript
    {
        public void OnWorkflowScriptExecute(Application app, WorkflowEventArgs args)
        {
            string docId = args.Document.EForm.Fields.Find("Related Doc Id").Value;
            string tableName = args.Document.EForm.Fields.Find("Search Table").Value;
            using (var conn = (OdbcConnection) app.Configuration.GetConnection("MyDbConnection"))
            {
                conn.Open();
                var builder = new OdbcCommandBuilder();
                string sanitizedTableName = builder.QuoteIdentifier(tableName);
                string sql = "SELECT keyvaluechar FROM " + sanitizedTableName + " WHERE itemnum = ?";
                using (var cmd = new OdbcCommand(sql, conn))
                {
                    cmd.Parameters.Add(new OdbcParameter
                    {
                        OdbcType = OdbcType.BigInt,
                        Value = long.Parse(docId)
                    });
                    var reader = cmd.ExecuteReader();
                    while (reader.HasRows && reader.Read())
                    {
                        // Do something with the data
                    }
                }
                conn.Close();
            }
        }
    }
}
```

```
// OLEDB Data Sanitation Example

using System.Data.OleDb;
using Hyland.Unity;

namespace UnityScripts
{
    public class OleDbParameterizationExample : IWorkflowScript
    {
        public void OnWorkflowScriptExecute(Application app, WorkflowEventArgs args)
        {
            string docId = args.Document.EForm.Fields.Find("Related Doc Id").Value;
            string tableName = args.Document.EForm.Fields.Find("Search Table").Value;
            using (var conn = (OleDbConnection) app.Configuration.GetConnection("MyDbConnection"))
            {
                conn.Open();
                var builder = new OleDbCommandBuilder();
                string sanitizedTableName = builder.QuoteIdentifier(tableName);
                string sql = "SELECT keyvaluechar FROM " + sanitizedTableName + " WHERE itemnum = @docId";
                using (var cmd = new OleDbCommand(sql, conn))
                {
                    cmd.Parameters.Add(new OleDbParameter
                    {
                        ParameterName = "docId",
                        OleDbType = OleDbType.BigInt,
                        Value = long.Parse(docId)
                    });
                    var reader = cmd.ExecuteReader();
                    while (reader.HasRows && reader.Read())
                    {
                        // Do something with the data
                    }
                }
                conn.Close();
            }
        }
    }
}
```

```
// MySQL Data Sanitation Example

using Hyland.Unity;
using MySql.Data.MySqlClient;

namespace UnityScripts
{
    public class MySqlParameterizationExample : IWorkflowScript
    {
        public void OnWorkflowScriptExecute(Application app, WorkflowEventArgs args)
        {
            string docId = args.Document.EForm.Fields.Find("Related Doc Id").Value;
            string tableName = args.Document.EForm.Fields.Find("Search Table").Value;
            using (var conn = (MySqlConnection) app.Configuration.GetConnection("MyDbConnection"))
            {
                conn.Open();
                var builder = new MySqlCommandBuilder();
                string sanitizedTableName = builder.QuoteIdentifier(tableName);
                string sql = "SELECT keyvaluechar FROM " + sanitizedTableName + " WHERE itemnum = @docId";
                using (var cmd = new MySqlCommand(sql, conn))
                {
                    cmd.Parameters.Add(new MySqlParameter
                    {
                        ParameterName = "docId",
                        MySqlDbType = MySqlDbType.Int64,
                        Value = long.Parse(docId)
                    });
                    var reader = cmd.ExecuteReader();
                    while (reader.HasRows && reader.Read())
                    {
                        // Do something with the data
                    }
                }
                conn.Close();
            }
        }
    }
}
```

```
// Common Data Sanitation Example

using System.Data;
using System.Data.Common;
using Hyland.Unity;

namespace UnityScripts
{
    public class CommonDbParameterizationExample : IWorkflowScript
    {
        public void OnWorkflowScriptExecute(Application app, WorkflowEventArgs args)
        {
            string docId = args.Document.EForm.Fields.Find("Related Doc Id").Value;
            string tableName = args.Document.EForm.Fields.Find("Search Table").Value;
            using (var conn = app.Configuration.GetConnection("MyDbConnection"))
            {
                conn.Open();
                var factory = DbProviderFactories.GetFactory(conn);
                var builder = factory.CreateCommandBuilder();
                string sanitizedTableName = builder.QuoteIdentifier(tableName);
                string sql = "SELECT keyvaluechar FROM " + sanitizedTableName + " WHERE itemnum = @docId";
                using (var cmd = conn.CreateCommand())
                {
                    cmd.CommandText = sql;
                    var param = cmd.CreateParameter();
                    param.ParameterName = "docId";
                    param.DbType = DbType.Int64;
                    param.Value = long.Parse(docId);
                    cmd.Parameters.Add(param);
                    var reader = cmd.ExecuteReader();
                    while (reader.HasRows && reader.Read())
                    {
                        // Do something with the data
                    }
                }
                conn.Close();
            }
        }
    }
}
```